ML-Exp 7: Ensemble Learning For House Price Prediction.

Student Name: REUEL ROLLAND GOMES

Roll No.: 23 Batch: 1 Date: 04-03-2026

```
import time
import pandas as pd
from sklearn.datasets import fetch_california_housing
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error
```

```
data = fetch_california_housing()
print(data.keys())
```

```
dict_keys(['data', 'target', 'frame', 'target_names', 'feature_names', 'DESCR'])
```

```
X = pd.DataFrame(data.data, columns=data.feature_names)
y = data.target
print(X.head())
print("Samples:", len(X))
print(y[:10])
```

```
   MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  Latitude  \
0  8.3252      41.0  6.984127   1.023810       322.0  2.555556     37.88
1  8.3014      21.0  6.238137   0.971880      2401.0  2.109842     37.86
2  7.2574      52.0  8.288136   1.073446       496.0  2.802260     37.85
3  5.6431      52.0  5.817352   1.073059       558.0  2.547945     37.85
4  3.8462      52.0  6.281853   1.081081       565.0  2.181467     37.85

   Longitude
0    -122.23
1    -122.22
2    -122.24
3    -122.25
4    -122.25
Samples: 20640
[4.526 3.585 3.521 3.413 3.422 2.697 2.992 2.414 2.267 2.611]
```

```
X.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 8 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   MedInc      20640 non-null  float64
 1   HouseAge    20640 non-null  float64
 2   AveRooms    20640 non-null  float64
 3   AveBedrms   20640 non-null  float64
 4   Population  20640 non-null  float64
 5   AveOccup    20640 non-null  float64
 6   Latitude    20640 non-null  float64
 7   Longitude   20640 non-null  float64
dtypes: float64(8)
memory usage: 1.3 MB
```

```
X.describe()
```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude
count	20640.000000	20640.000000	20640.000000	20640.000000	20640.000000	20640.000000	20640.000000	20640.000000
mean	3.870671	28.639486	5.429000	1.096675	1425.476744	3.070655	35.631861	-119.569704
std	1.899822	12.585558	2.474173	0.473911	1132.462122	10.386050	2.135952	2.003532
min	0.499900	1.000000	0.846154	0.333333	3.000000	0.692308	32.540000	-124.350000
25%	2.563400	18.000000	4.440716	1.006079	787.000000	2.429741	33.930000	-121.800000
50%	3.534800	29.000000	5.229129	1.048780	1166.000000	2.818116	34.260000	-118.490000
75%	4.743250	37.000000	6.052381	1.099526	1725.000000	3.282261	37.710000	-118.010000
max	15.000100	52.000000	141.909091	34.066667	35682.000000	1243.333333	41.950000	-114.310000

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
tree = DecisionTreeRegressor()
start = time.perf_counter()
tree.fit(X_train, y_train)
tree_time = time.perf_counter() - start
pred_tree_test = tree.predict(X_test)
err_tree_test = mean_absolute_error(y_test,pred_tree_test)
pred_tree_train = tree.predict(X_train)
err_tree_train = mean_absolute_error(y_train, pred_tree_train)
print("Decision Tree Train MAE:", err_tree_train)
print("Decision Tree Test MAE:", err_tree_test)
print("Decision Tree Time", tree_time)
```

```
Decision Tree Train MAE: 4.206368242523607e-17
Decision Tree Test MAE: 0.45588703488372095
Decision Tree Time 0.22905589999982112
```

```
rf = RandomForestRegressor(n_estimators=100)
start = time.perf_counter()
rf.fit(X_train, y_train)
rf_time = time.perf_counter() - start
pred_rf_test = rf.predict(X_test)
err_rf_test = mean_absolute_error(y_test,pred_rf_test)
pred_rf_train = rf.predict(X_train)
err_rf_train = mean_absolute_error(y_train, pred_rf_train)
print("Random Forest Train MAE:", err_rf_train)
print("Random Forest Test MAE:", err_rf_test)
print("Random Forest Time", rf_time)
```

```
Random Forest Train MAE: 0.12209821948280077
Random Forest Test MAE: 0.3270506751211242
Random Forest Time 12.762425899999926
```

```
Gb = GradientBoostingRegressor(n_estimators=300, learning_rate=0.1, max_depth=5)
start = time.perf_counter()
Gb.fit(X_train, y_train)
Gb_time = time.perf_counter() - start
pred_Gb_test = Gb.predict(X_test)
err_Gb_test = mean_absolute_error(y_test,pred_Gb_test)
pred_Gb_train = Gb.predict(X_train)
err_Gb_train = mean_absolute_error(y_train, pred_Gb_train)
print("Gradient Boosting Train MAE:", err_Gb_train)
print("Gradient Boosting Test MAE:", err_Gb_test)
print("Gradient Boosting Time", Gb_time)
```

```
Gradient Boosting Train MAE: 0.2296832479303222
Gradient Boosting Test MAE: 0.3065927741253858
Gradient Boosting Time 21.894609200000104
```

```
print("\n== Comparison For Test MAE ==")
print("Decision Tree Test MAE:", err_tree_test)
print("Random Forest Test MAE:", err_rf_test)
print("Gradient Boosting Test MAE:", err_Gb_test)
```

```

== Comparison For Test MAE ==
Decision Tree Test MAE: 0.45588703488372095
Random Forest Test MAE: 0.3270506751211242
Gradient Boosting Test MAE: 0.3065927741253858
```

```
print("\n== Comparison For Train MAE ==")
print("Decision Tree Train MAE:", err_tree_train)
print("Random Forest Train MAE:", err_rf_train)
print("Gradient Boosting Train MAE:", err_Gb_train)
```

```

== Comparison For Train MAE ==
Decision Tree Train MAE: 4.206368242523607e-17
Random Forest Train MAE: 0.12209821948280077
Gradient Boosting Train MAE: 0.2296832479303222
```

```
print("\n== Comparison For Time ==")
print("Decision Tree Time:", tree_time)
print("Random Forest Time:", rf_time)
print("Gradient Boosting Time:", Gb_time)
```

```

== Comparison For Time ==
Decision Tree Time: 0.22905589999982112
Random Forest Time: 12.762425899999926
Gradient Boosting Time: 21.894609200000104
```